COSC419/519:DEEP LEARNING

Linear Models

Part-1

Dr. Mohamed Shehata

Computer Science

University of British Columbia

UBC OKANAGAN

Machine Learning

Let's say we have experience E (e.g., played games of checkers) with respect to tasks T (e.g., playing checkers), and performance measure P (e.g., probability that the computer will win the next game).

If the performance P at tasks T improves with more experience E, this is the definition of ***machine learning.***

"Tom Mitchell"

Tom Mitchell
1997

Learning from experience to enhance performance

EXAMPLE:

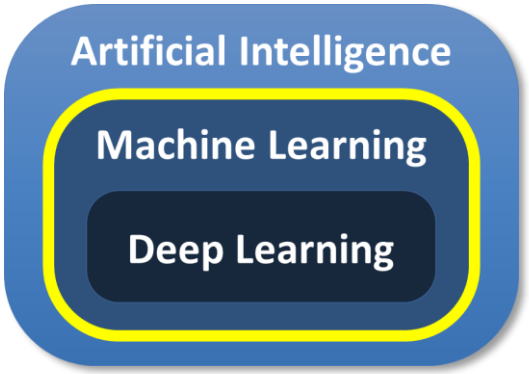
E: the experience of playing thousands of games

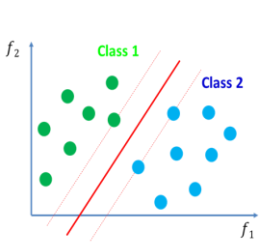
T: playing Checkers

P: the fraction of games the algorithm wins against human opponents

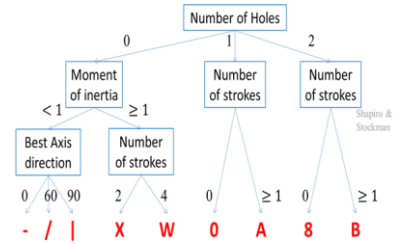
Machine Learning

- Aspects of machine learning:
  - Types of Learning
    - **Supervised learning**
    - **Unsupervised learning**
    - More advanced learning techniques:
      - Semi-supervised learning
      - Self-supervised learning (SOTA)
      - Reinforcement learning
      - Few-shot learning
      - Continual learning
      - Federated learning
  - Types of problems.
    - **Regression problems**
    - **Classification problems**

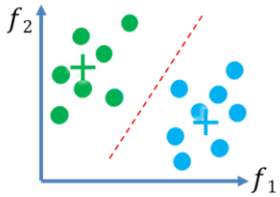




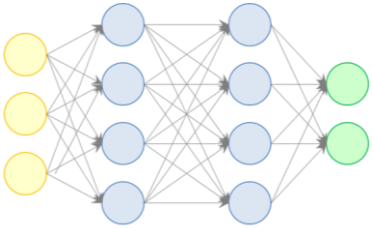
SVM



Decision Trees

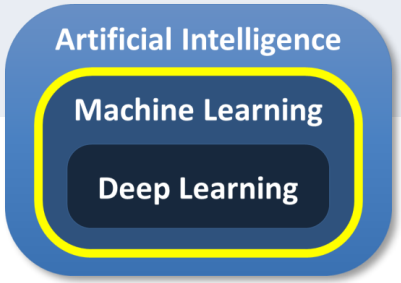


K-mean clustering

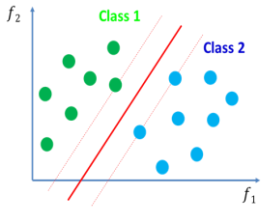


Artificial Neural Networks

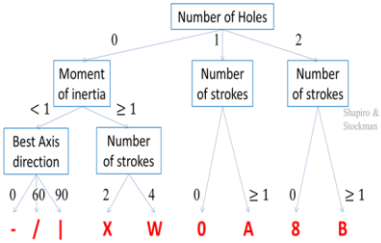
Machine Learning Training



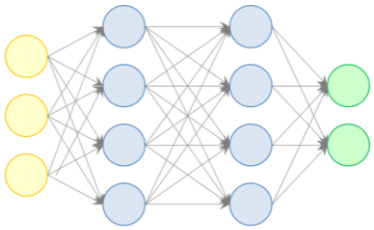
- Generally, machine learning is <u>trained using data points</u> :
  - Supervised learning
    - Machine trains on data and expected output (i.e. data is labeled)
    - Examples of supervised learning models:
      - SVM
      - Decision Trees
      - ANN
  - Unsupervised learning.
    - Unlabeled data (no feedback based on the prediction results)
    - Examples of unsupervised learning models:
      - K-mean
  - Self-Supervised learning (SSL)
    - Trains models using unlabeled data to generate supervisory signals
    - For training, there are mainly two stages:
      - Pretext task
      - Downstream tasks (application specific, finetune the pretext mode)
    - Examples:
      - SimCLR (Contrastive learning)
      - DinoV2 (Non-Contrastive learning)



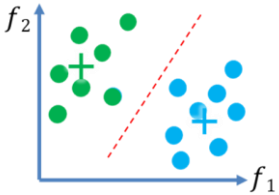
SVM



Decision Trees



Artificial Neural Networks



K-mean clustering

Contrastive learning uses both positive and negative examples
Non-contrastive learning Uses only positive examples

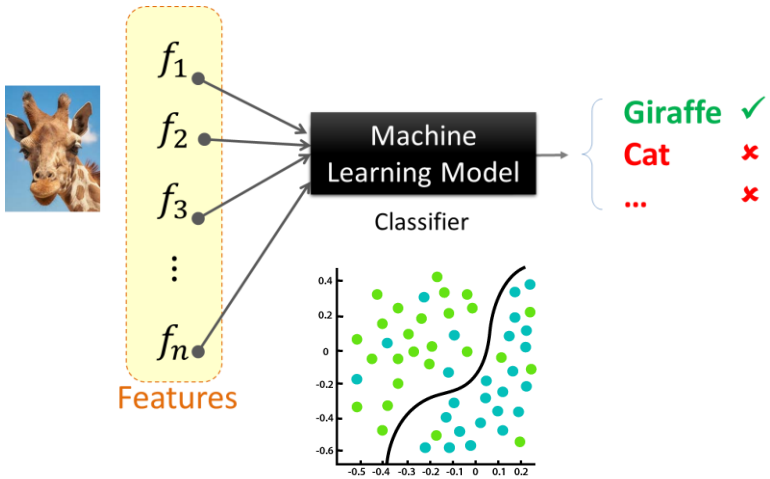
Motivating Problem

- We want to predict the share price of a company, in order to decide when to sell/buy shares
- We have access to the revenue of the company, number of employees, taxes, oil price, etc.
- How can we predict the share price?
- Mathematical modeling:
  - Using mathematical analysis, find a relation between the input variables (revenue, number of employees, taxes, oil price, etc.) and the output variable (share price)
- Learning from data:
  - Using a large dataset of input revenue, number of employees, taxes, oil price, etc. and share prices of many companies, learn a relation between inputs variables and the output variable
  - This is the machine learning approach

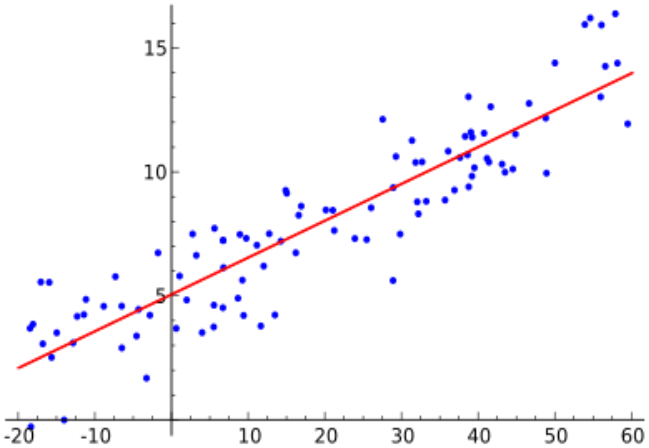
Linear Models

Linear Models

- One of the fundamental building blocks in machine/deep learning are the linear models, where the model's output is calculated based on a linear function. There are two main applications of linear models:
  - Regression
  - Classification



Example: Classification



Example: Regression

Regression vs Classification

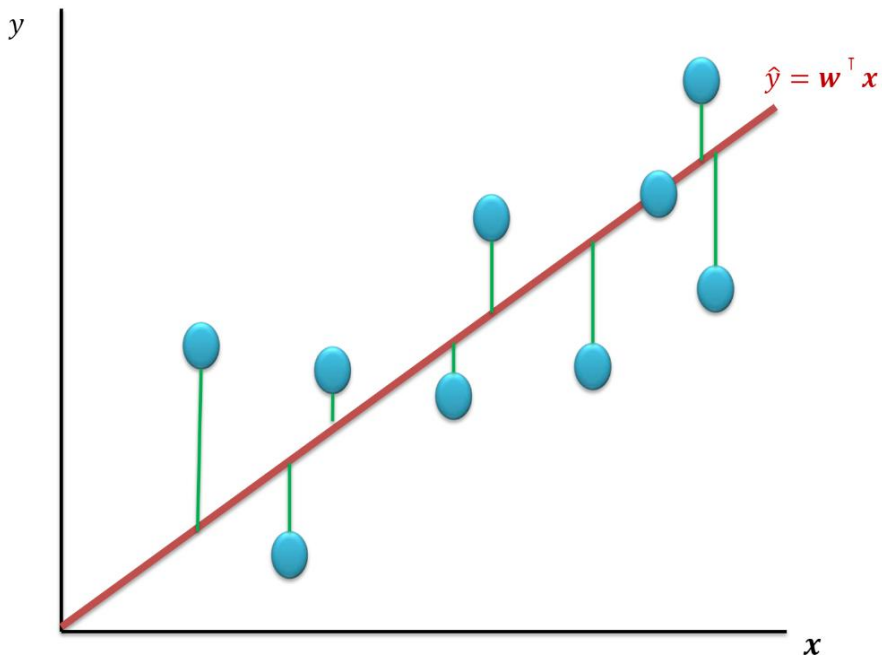
1) Regression problems:
   - we are trying to map input variables to some continuous function.
     - Example: given data about the size of houses on the real estate market, try to predict their price.
2) Classification problem:
   - we are trying to predict results in a *discrete output*. That is, we are trying to map input variables into discrete categories.
     - Binary Classification: predict a binary label (e.g. cat vs. non-cat)
     - Multiway classification: predict a discrete label (e.g. object category [for example, cat], from a list [for example, cat, dog, horse, bird])
   - For example, an SVM (Support Vector Machine) learns the boundary between data of two (or more) categories

Regression

Reading: Chapter 3, Machine Learning Refined

Linear Regression

- Linear regression solves a regression problem.
- The goal is to build a system that can take a vector $x \in \mathbb{R}^n$ as input and predict the value of a scalar $y \in \mathbb{R}$ as its output.
- The output of linear regression is a linear function.



Problem setup

- Given a dataset of pairs $(\boldsymbol{x}_1^{train}, y_1^{train}), (\boldsymbol{x}_2^{train}, y_2^{train}), \ldots$ where:
  - $(\boldsymbol{x}^{train}, y^{train})$:
    - $\boldsymbol{x}$ is the input vector **(bolded means vector)**
    - y is the output or target (scalar)

- Then, the experience E is the training of the model using $\{(\boldsymbol{x}_p^{train}, y_p^{train})\}_{p=1}^{P}$ where P is all the input/output observation pairs
  - In general, each input $\boldsymbol{x}_p^{train}$ may be a scalar value or a column vector of length N+1 ((N+1) x 1 vector)
  - The "model" can be represented by a set of weights W which is a column vector of the same length
  - N is the number of features in each input vector

$$x_p = \begin{bmatrix} 1 \\ x_{1,p} \\ x_{2,p} \\ \vdots \\ x_{N,p} \end{bmatrix}_{(N+1)\times 1} \qquad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}_{(N+1)\times 1}$$

- We augment $\boldsymbol{x}_p^{train}$ by adding 1 as as the first dimension to correspond for the bias term in w

- When dealing with N dimensional input we also include a **bias** term along with N slope weights, and add a 1 as the first element of each $\boldsymbol{x}_p$ vector. This gives:

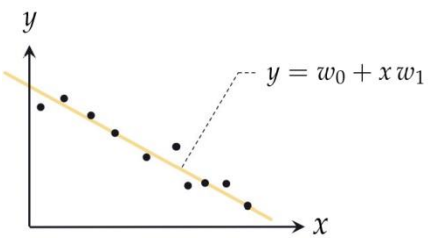
$$\boldsymbol{x}^{\boldsymbol{T}}\boldsymbol{w} = w_0 + x_{1,p}w_1 + x_{2,p}w_2 + \ldots + x_{N,p}w_N \approx y_p, \qquad p = 1,2,\ldots,P$$

  - Each value in the *input* is referred to in the jargon of machine learning as a *feature* or *input feature*. Each weight in the *weight vector* W is called a weight or parameter.

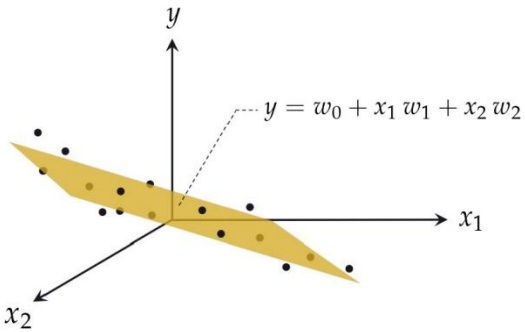
Low Dimensional Example

- In simple instances (N=1), the input is scalar-valued (the output of a model is always scalar-valued), and hence the linear regression problem is, geometrically speaking, one of fitting a line to the associated scatter of data points in a 2-dimensional space, modeled as:

$$\therefore w_0 + x_{1,p} w_1 \approx y_p, \qquad p = 1,2,\ldots,P$$



- When N=1, this means that the input $\boldsymbol{x}$ will have one scalar value (top figure) which forms a line.
- If N=2, then input $\boldsymbol{x}$ will have two values $x_1 and\ x_2$ (bottom figure), which will form a hyper plane.
  - For example, a specific input p of a data pair $\boldsymbol{x}_p = (x_{1,p}, x_{2,p})$ where $x_{1,p}$ is the nose size of an animal and $x_{2,p}$ is the ear size



The Bias

- Motivation: Models need a bias term w_0 to handle cases where the output should be non-zero even when all features are zero.
- Step 1: Augment features with 1
- Step 2: Include bias in weights
- Step 3: Unified model $\hat{y} = \boldsymbol{x}^T w$
- Target: we want $\hat{y}$ and y to be the same value, but there will always be an error.
- How can we measure this error and hence quantify the performance of our model?

- Hence, The task T: predict y from $\boldsymbol{x}$ by using the linear model

$$\hat{y} = \boldsymbol{x}^T w$$

  - This is also called the regression model with input $\boldsymbol{x}$ and output $\hat{y}$
  - $\hat{y}$ is the value that our model predicts (we want to be $\approx y$).
  - $\boldsymbol{x} \in \mathbb{R}^N$ is the input data
  - $y \in \mathbb{R}$ is the ground truth output value
  - $\boldsymbol{w} \in \mathbb{R}^N$ is the weight/parameters vector of the model
  - $\boldsymbol{w}$ controls the behavior of the system
  - Settings of the parameters are called hypotheses

Notation

- For a SINGLE data point (p-th point):

$$x_p = \begin{bmatrix} 1 \\ x_{1,p} \\ x_{2,p} \\ \vdots \\ x_{N,p} \end{bmatrix}_{(N+1)\times 1} \qquad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}_{(N+1)\times 1}$$

$$\hat{y}_p = x_p^T\, w \text{ (transpose x_p to make it a row vector)}$$
$$= w_0 + x_{1,p}w_1 + x_{2,p}w_2 + \ \ldots + x_{N,p}w_N$$

- For ALL data points (entire dataset):

$$X = \begin{bmatrix} 1 & x_{1,1} & \cdots & x_{N,1} \\ \vdots & & \ddots & \vdots \\ 1 & x_{1,P} & \cdots & x_{N,P} \end{bmatrix}_{P\times(N+1)} \qquad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}_{(N+1)\times 1}$$

$$\hat{Y} = X\, w \text{ (No transpose on X)}$$

predictions

Ground truth

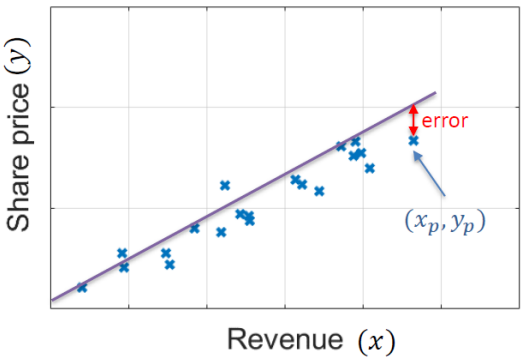
$$\text{Where: } \hat{Y} = \begin{bmatrix} \hat{Y}_1 \\ \\ \\ \hat{Y}_P \end{bmatrix}_{P\times 1} \quad \text{AND} \quad Y = \begin{bmatrix} Y_1 \\ \\ \\ Y_P \end{bmatrix}_{P\times 1}$$

- Capital letter means entire dataset while small letter means single datapoint

Performance Example

- Consider a pair $(\boldsymbol{x}_p, y_p)$ and the model outputs $\hat{y}$
- Consider N=1 (only one input feature)
- In general, $y_p \neq \hat{y}$ (there is always error)
- We define a performance measure:
  - The square error for point p as

    $(\hat{y} - y_p)^2$ (this error is always positive)



- Example: Consider $\boldsymbol{w}$=(w_0,w_1)=(100,10) and (x_p,y_p)=(100,1000) (a company with a revenue of 100M and share price of 1000)
  - $\hat{y}$ =w_0+$w_1 x_p$=100+(10)*(100)=1100
  - $(\hat{y} - y_p)^2$ = $(1100 - 1000)^2$ =10000

Performance using MSE

- In previous slide we measured the square error during training (because we have the ground truth)
- In general. measure the performance of the model using mean squared error (MSE) on $(\boldsymbol{x}, \boldsymbol{y})$

$$MSE = \frac{1}{P}\sum_{i}(\hat{y} - y)_i^2$$

$$= \frac{(\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + \cdots + (\hat{y}_P - y_P)^2}{P}$$

- The MSE is also called the cost function

$g(w) = J(w) = MSE$ (different notations depending on textbook)

- The cost function is also used to measure the performance of the model using an input vector of unseen data called $\boldsymbol{x}^{(test)}$ and the vector of regression targets as $y^{(test)}$
- **NOW:** P, x_p , and y_p are all given. Thus the cost function is a function of $\boldsymbol{w}$ only
- We need to find $\boldsymbol{w}$ by minimizing the MSE during the training and then testing this during testing
- Two solutions:
  - Direct solution: minimizing the MSE_{train} where its gradient is zero by solving its normal equations (**see next slide**). This is not suitable for many cases.
  - Iterative solution: use an iterative method to update the model performance in each iteration **(More on this later)**

Direct Solution

- The least square cost function is a convex quadratic function
- A single Newton step can perfectly minimize it (take the derivative and set it to zero):

Given $\hat{y} = \boldsymbol{x}^T w$ and $MSE = \frac{1}{P}\sum_{p=1}^{P} \left(\hat{y}_p - y_p\right)_i^2$

$$\nabla_w \left(MSE_{train}\right) = 0$$

- This is known as the normal equation.
- This is sometimes referred to as minimizing via solving the normal equations

$$w_{(N+1)\times 1} = \left(X^{(train)\,T} X^{(train)}\right)^{-1} X^{(train)\,T} Y^{(train)}$$

Where: $X = \begin{bmatrix} 1 & x_1^T \\ 1 & x_2^T \\ 1 & x_3^T \\ \vdots & \vdots \\ 1 & x_P^T \end{bmatrix}_{P\times(N+1)}$ $\quad \hat{Y} = \begin{bmatrix} \hat{Y}_1 \\ \\ \hat{Y}_P \end{bmatrix}_{P\times 1}$ $\quad Y = \begin{bmatrix} Y_1 \\ \\ Y_P \end{bmatrix}_{P\times 1}$

Example

- We have N=1 (only one input feature)
- $data\ points = [(1,2),(2,3),(3,4),(4,5)]$

$$\boldsymbol{x} = \{\mathbf{1},\mathbf{2},\mathbf{3},\mathbf{4}\} \Rightarrow \boldsymbol{X} = \begin{bmatrix} \mathbf{11} \\ \mathbf{12} \\ \mathbf{13} \\ \mathbf{14} \end{bmatrix}$$ **(size PxN+1=4x2)**

The prediction vector is hence

$$\widehat{\boldsymbol{Y}} = \boldsymbol{X}\,\boldsymbol{w} = \begin{bmatrix} \mathbf{11} \\ \mathbf{12} \\ \mathbf{13} \\ \mathbf{14} \end{bmatrix} \begin{bmatrix} \boldsymbol{w_0} \\ \boldsymbol{w_1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{w_0} + \mathbf{1}\boldsymbol{w_1} \\ \boldsymbol{w_0} + \mathbf{2}\boldsymbol{w_1} \\ \boldsymbol{w_0} + \mathbf{3}\boldsymbol{w_1} \\ \boldsymbol{w_0} + \mathbf{4}\boldsymbol{w_1} \end{bmatrix}$$

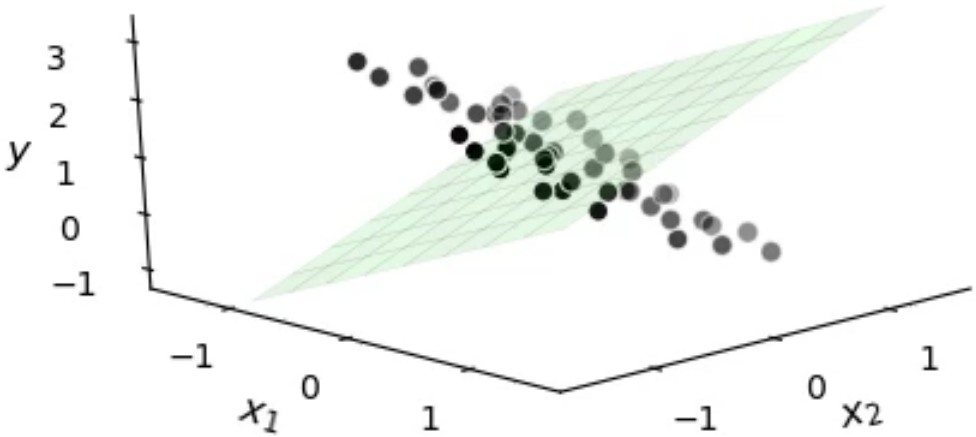
Sample Code for Direct Solution

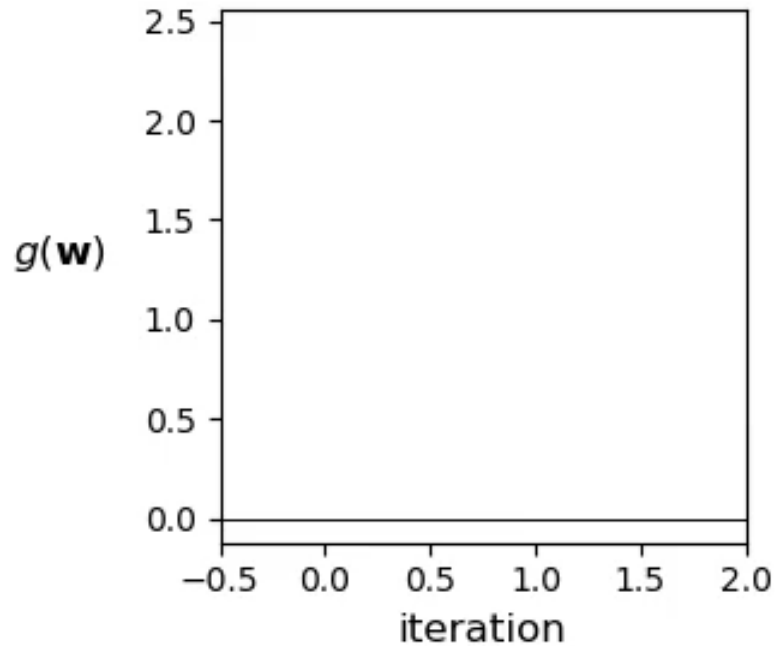
```
import numpy as np
from numpy.linalg import inv

# Input features
X= np.array([[1, 1], [1, 2], [1, 3], [1, 4]])

# Target values
Y = np.array([[2], [3], [4], [5]])

# Transpose of X
X_transpose = X.T

# Compute X^T * X
X_transpose_X = np.dot(X_transpose, X)

# Compute X^T * y
X_transpose_Y = np.dot(X_transpose, Y)

# Calculate the inverse of X_transpose_X
X_transpose_X_inv=inv(X_transpose_X);

# Solve the normal equation
w=np.dot(X_transpose_X_inv, X_transpose_Y)

# print the weights
print("w[0] = bias w0:", w[0, 0])
print("w[1] = weight w1:", w[1, 0])

# Input features for testing (1,1) and (1,4)
X_test = np.array([[1,1], [1,4]])
predictions = np.dot(X_test, w)
print(predictions)
```

$$w_{(N+1)\times 1} = \left(X^{(train)\,T} X^{(train)}\right)^{-1} X^{(train)\,T} Y^{(train)}$$

X: The first column represents the bias term (intercept).The second column is the only feature (one independent variable).
This means **X is a 2D feature space**, and the regression equation is: y= w0+w1x1

Visual View on Large Dataset





- N=2
- g(w) is the cost function MSE
- Green is the initial weights and then the red is the final weights after solving the normal equation

Exercise at Home

Given $MSE = \frac{1}{P}\sum_i (\hat{y} - y)_i^2$

Prove that the solution to

$$\nabla_w \left(MSE_{train}\right) = 0$$

is $$w = \left(X^{(train)\,T} X^{(train)}\right)^{-1} X^{(train)\,T} Y^{(train)}$$

where

Where: $X = \begin{bmatrix} 1 & x_1^T \\ 1 & x_2^T \\ 1 & x_3^T \\ \vdots & \vdots \\ 1 & x_P^T \end{bmatrix}_{P\times(N+1)}$ $\hat{Y} = \begin{bmatrix} \hat{Y}_1 \\ \\ \hat{Y}_P \end{bmatrix}_{P\times 1}$ $Y = \begin{bmatrix} Y_1 \\ \\ Y_P \end{bmatrix}_{P\times 1}$

Binary Classification (Logistic Regression)

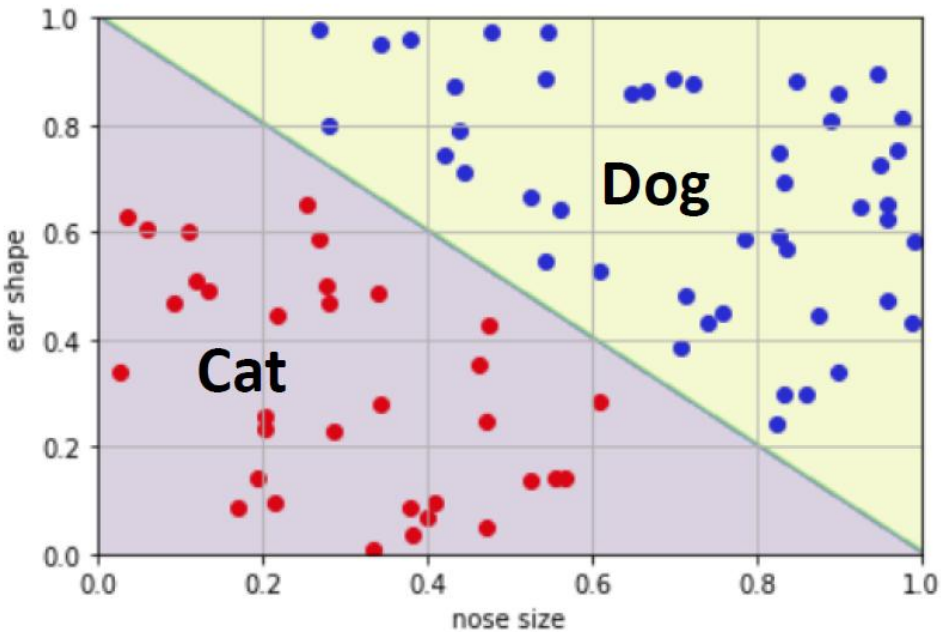
Reading: Chapter 6, Machine Learning Refined

Two-Class Classification

- Key difference between regression and classification:
  - Regression: output takes on a continuous value (i.e. *real* values)
  - Classification: output takes on discrete values (e.g. {0,1})

Examples:

- Object detection – face or not a face
- Medical diagnosis – ill or not
- Fault detection – correct or faulty

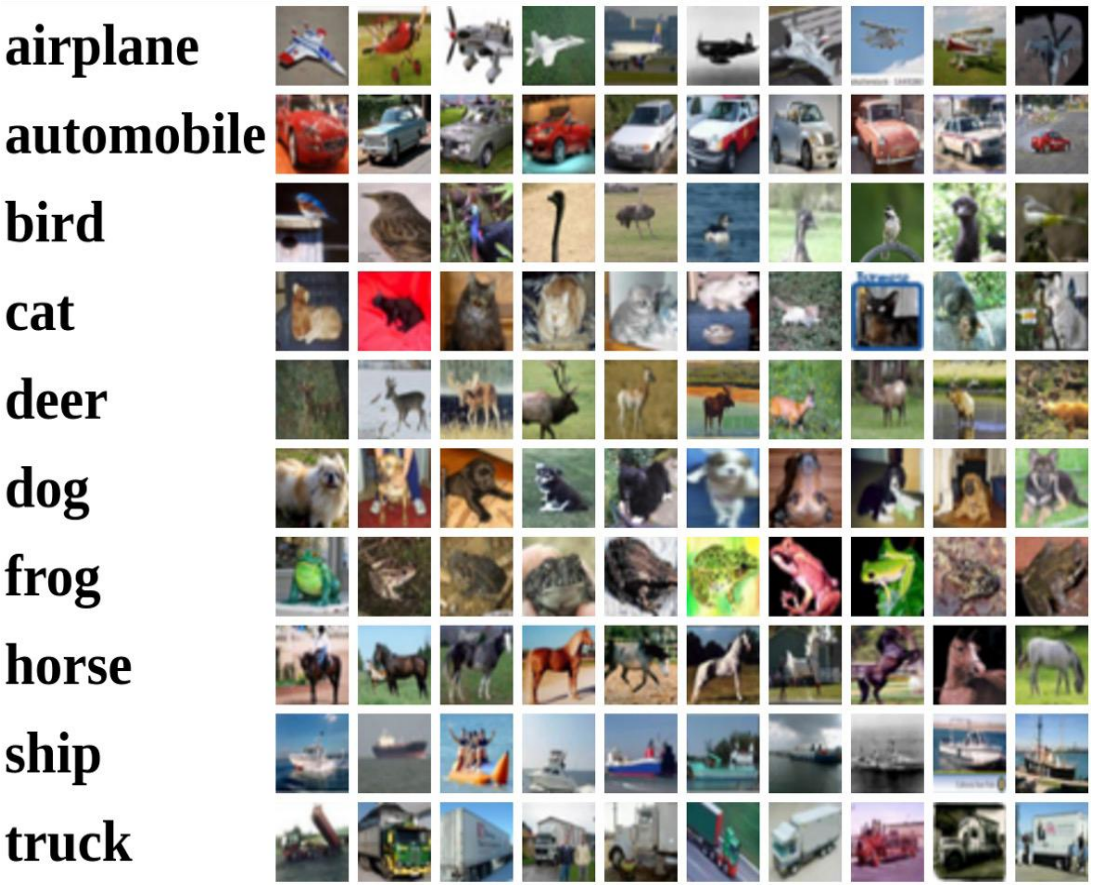


Machine Learning: Data-Driven Approach

1. Collect a dataset (e.g., images and their labels)
2. Use Machine Learning algorithms to train a classifier
3. Evaluate the classifier on new images

Example Dataset: CIFAR10

- 10 classes
- 50,000 training images
- 10,000 testing images



General Setup

Remember: C=2 Classes

Input $\{(\boldsymbol{x}_p, y_p)\}_{p=1}^{P}$

N=2: nose, ear

e.g., $(x_{1,p}, x_{2,p})$ of the pth data point

Output:

class $\widehat{y_p} \in \{0,1\}$,

$p = 1,2, \ldots, P$

Classes: C =2 (cat, dog)

Data set

Then during testing

New data at testing: nose-ear (x_1, x_2)

Learning (or training)

Classification model $f(\mathrm{w}, x_p)$

Tuning the parameters ($\boldsymbol{w}$) of the model

Predicted class

$\widehat{y_p} \rightarrow 0 =$ cat

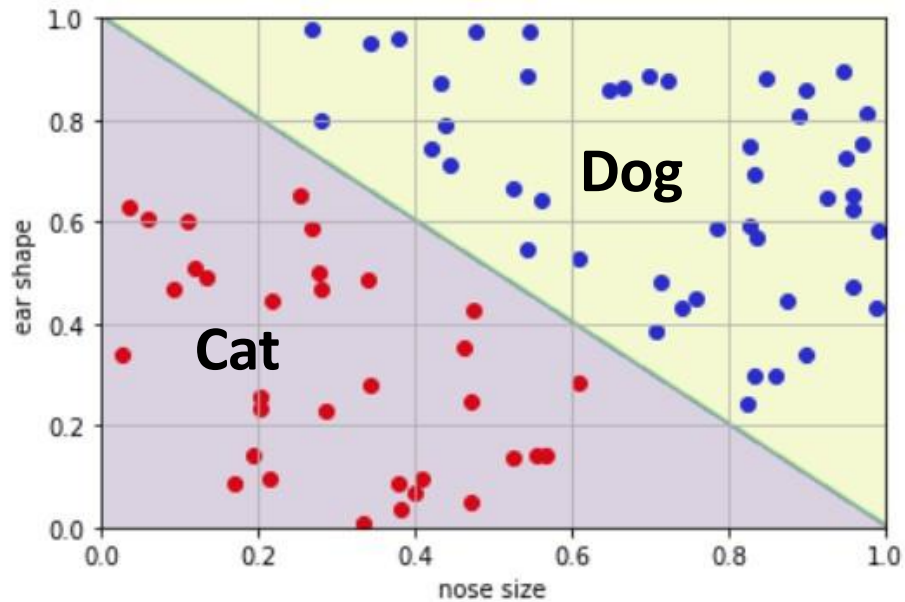
$\widehat{y_p} \rightarrow 1 =$dog

($\widehat{y_p}$ Scalar, 1 probability)

Data input/output pairs

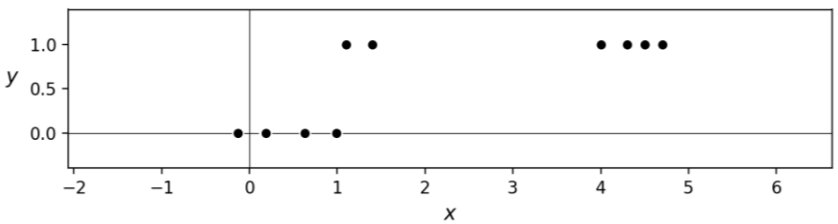
- Data: P input/output pairs $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_P, y_P)$
  - $\boldsymbol{x}_p = (x_{1,p}, x_{2,p}, \ldots, x_{N,p})$ is an N-dimensional *feature vector for the pth data point*
  - y_p takes on two discrete values, often denoted 0 and 1
- y_p are often called *labels or ground truth*, and data points sharing the same label are called a *class*

- Example: (N=2)
  - $\boldsymbol{x}_p = (x_{1,p}, x_{2,p})$ where $x_{1,p}$ is the nose size and $x_{2,p}$ is the ear shape
    p is the pth data point
  - $\hat{y}_p = 0$ if sample p is a cat and $\hat{y}_p = 1$ if it is a dog

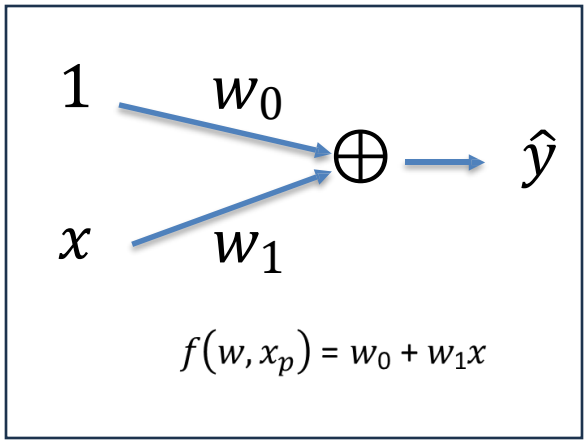


Linear Regression to Logistic Regression

- Let us simplify the problem by assuming we have only one feature N=1 (e.g., nose size) and still C=2



- Dataset: P data points $(x_1, y_1), (x_2, y_2), \ldots, (x_P, y_P)$
  - x_p: Input feature (nose size), N=1
  - y_p: ground truth (cat=0, dog=1)
- Task: Find an **Activation Function $f(w, x_p)$**
  Such that $y_p \approx \hat{y}_p = f(w, x_p)$



Linear regression model

- How do we modify the model activation $f(w, x_p) = w_0 + w_1 x$ to achieve this task?
  - It is obvious that simply fitting a line of the form $y=w_0+w_1x$ to such a dataset will result in an extremely subpar results.
  - The line by itself is simply too inflexible to account for the nonlinearity present in such data.
  - Such data needs to be fit with a step function, meaning, fit a discontinuous step function with a linear boundary

Activation Function: Step Regression

Use $\widehat{y_p} = \text{step}(w, x_p)$

When N=1, a linear model defining this boundary is just a line w_0+xw_1 composed with the step function step(·), i.e. the model looks like:

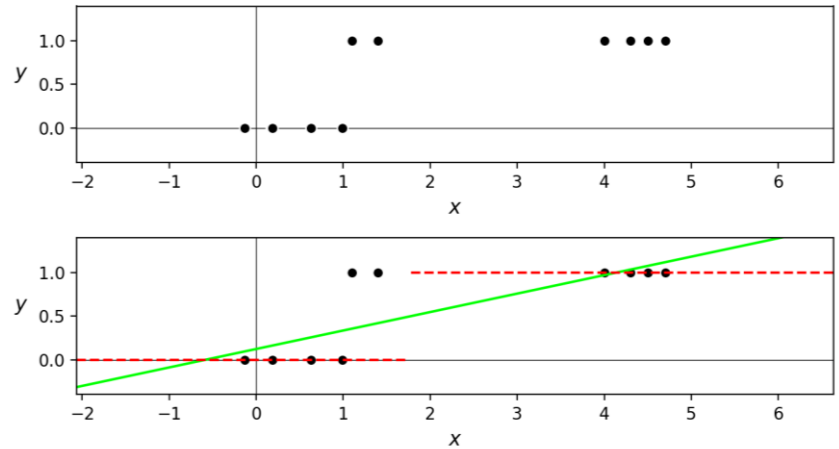
$$\widehat{y_p} = step(w_0 + xw_1)$$

where the step function is:

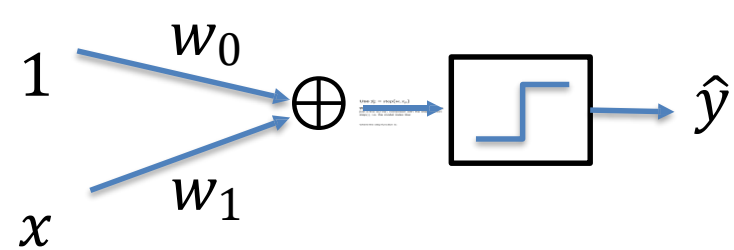
$$step(a) = \begin{cases} 1 \; if \; a \geq 0.5 \\ 0 \; if \; a < 0.5 \end{cases}$$

The linear boundary (vertical portion of the step function) between the two steps is defined by all points x where:

$$w_0 + x_1 w_1 = 0.5$$



linear fit (in green) as well as its composition with the step function (in dashed red) are shown along with the data in the bottom panel



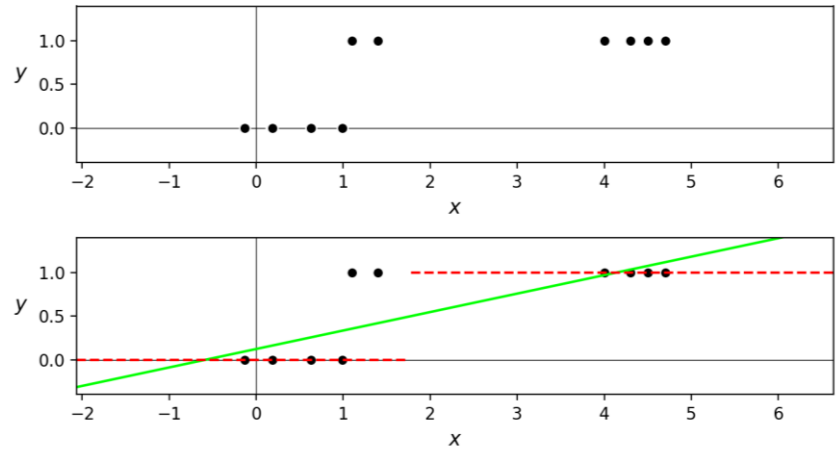
Note: The top dashed "1" segment **does not extend left** just to cover the two points where they have y=1 — those two points are simply **misclassified** by that choice of $w1$ and $w0$.

Step Regression

Use $\widehat{y_p} = \text{step}(w, x_p)$

More generally, with general N-dimensional input:

$$x_p = \begin{bmatrix} 1 \\ x_{1,p} \\ x_{2,p} \\ \vdots \\ x_{N,p} \end{bmatrix} \qquad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}$$

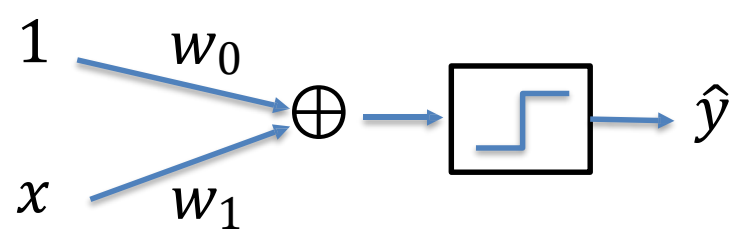


linear fit (in green) as well as its composition with the step function (in dashed red) are shown along with the data in the bottom panel

A corresponding step function is then simply the step(·) of the linear combination of x and w:

$$\widehat{y_p} = step\left(\overrightarrow{x_p}^T w\right)$$

and the linear boundary between the steps is defined by all points $\vec{x}$ where: $\overrightarrow{x_p}^T w = 0.5$



Finding the Parameters w

- How do we tune the w parameters properly?
- Use the MSE loss function (average of the square difference between prediction and ground truth):

$$MSE = \frac{1}{P}\sum_{1}^{P}(\widehat{y_p} - y_p)^2$$

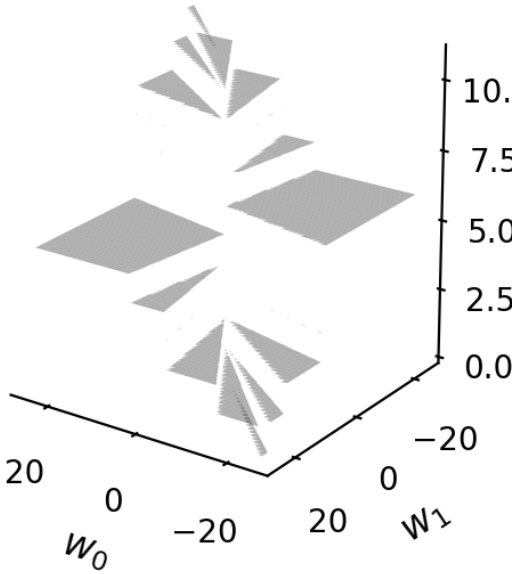
$$= \frac{1}{P}\sum_{1}^{P}(step(f(w, x_p)) - y_p)^2$$

$$= \frac{1}{P}\sum_{1}^{P}(step(\boldsymbol{x}_p^T\,\boldsymbol{w}) - y_p)^2$$

- We could then try to minimize this cost in order to recover weights that satisfy our desired equalities.
- If we can find a set of weights such that MSE=0 then all P equalities above hold true, otherwise some of them do not.

Finding the Parameters w

- However: Step function is not ‘nice’ for optimization
  - Not continuous (this Least Squares cost takes on only integer values)
  - Not differentiable. In other words, it is impossible to minimize with local optimization (e.g., gradient-based techniques), as at every point the function is completely flat.
  - Because of this, local optimization cannot easily take even a single step 'downhill' regardless of where they are initialized.
  - This problem is inherited from our use of the step function, itself a discontinuous step.



Activation Function: Sigmoid Regression

- Solution: Replace the step function with a sigmoid logistic function

$$\sigma(t) = \frac{1}{1+e^{-t}}$$

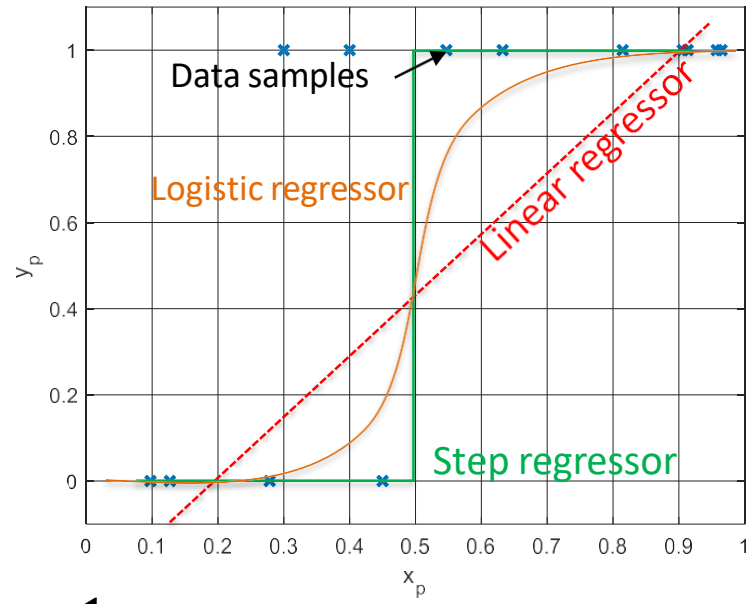
$$f(w,x) = \sigma(w_0 + w_1 x)$$

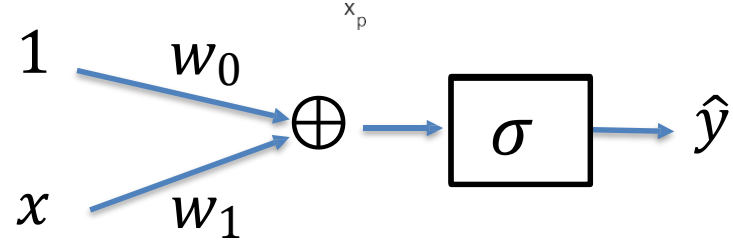
$$g(w) = \frac{1}{P}\sum_{1}^{P}(\sigma(\boldsymbol{x}_p^T \boldsymbol{w}) - y_p)^2$$

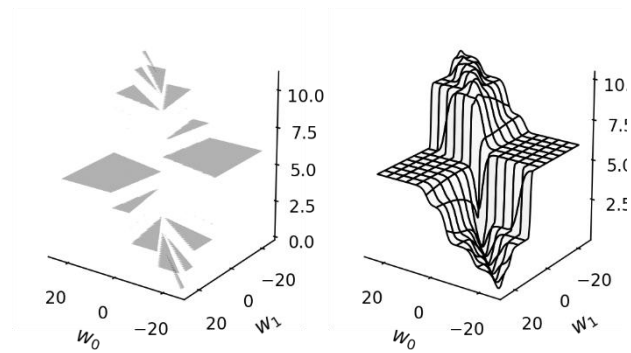
- This function is plotted to right of the lowest figure
  - Generally non-convex, but can use normalized Gradient Descent (NGD)
  - Too complex: We can simply employ a different and convex cost function
  - More on this later
- Note: We need to threshold $\hat{y}$ at 0.5

$\hat{y} > 0.5$ ➔ class 1

$\hat{y} \leq 0.5$ ➔ class 0







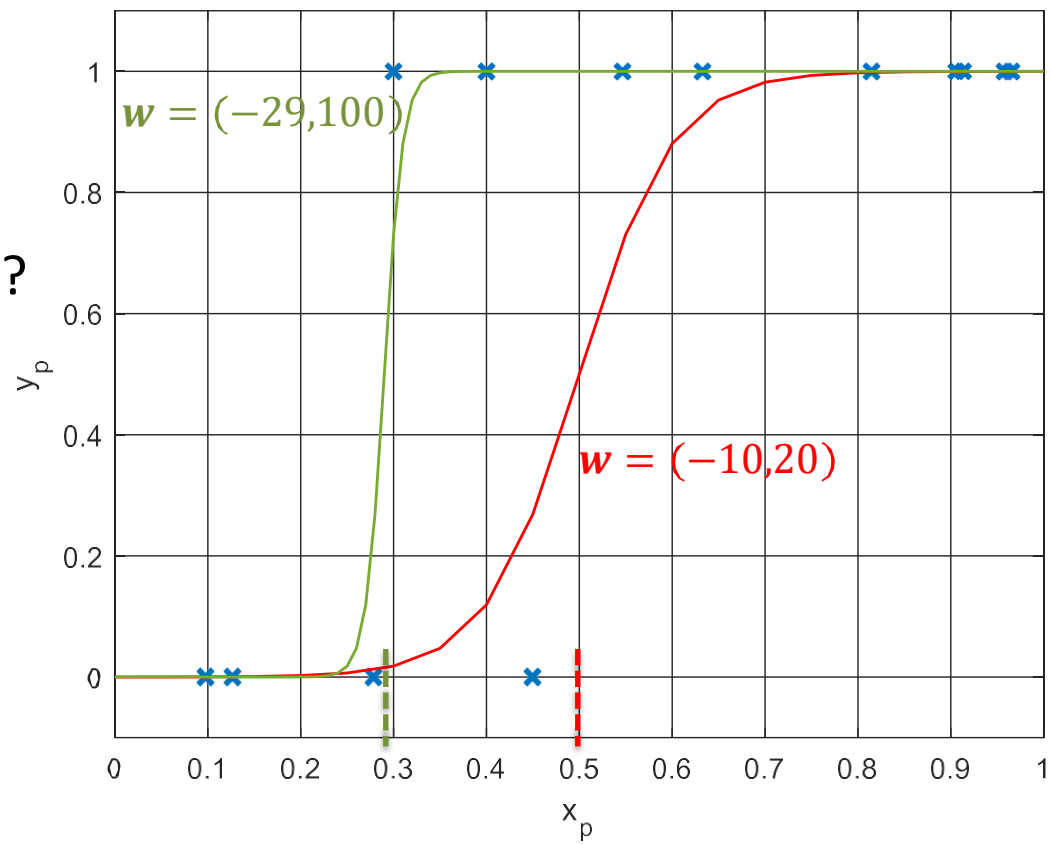
Example of sigmoid logistic function

This is an N=1 Example (one feature in the input)

- $f(w, x) = \sigma(w_0 + w_1 x)$
- How do we choose $\boldsymbol{w} = (w_0, w_1)$?
- Which of the two logistic regression models is better?

- We need to find the value of x where the model evaluates to 0.5

  - Model 1: $w = (-10,20)$ $\Rightarrow f_{\boldsymbol{w}}(x) = 0.5$ @ $x = 0.5$
  - Model 2: $w = (-29,100)$ $\Rightarrow f_{\boldsymbol{w}}(x) = 0.5$ @ $x = 0.29$



- By counting errors: Model 2 (1 error) is better than model 1 (2 errors)
- $\Rightarrow$ We need to define a cost function that quantifies errors, which we can then minimize to *train the model*

Loss: Cross-Entropy Error

- Consider a pair (x_p, y_p) and the model output as:

$$y_p \approx \hat{y} = f\left(w, x_p\right) = \sigma(x_p^T w) = \sigma(w_0 + w_1 x)$$

- In general, $y_p \neq f(\mathrm{w}, x_p)$
- In previous Sigmoid regression, we calculated the error using

$$g(w) = \frac{1}{P}\sum_{1}^{P}(\sigma(\boldsymbol{x}_p^T\,\boldsymbol{w}) - y_p)^2$$

- However, to overcome issues discussed before, we can measure the error for **one data point $\boldsymbol{p}$** using the Log Error (here we use the natural log) as follows:

$$g_p(w) = \begin{cases} -\log\left(\sigma\left(\boldsymbol{x}_p^T\,\boldsymbol{w}\right)\right) & if\ y_p = 1 \\ -\log\left(1 - \sigma\left(\boldsymbol{x}_p^T\,\boldsymbol{w}\right)\right) & if\ y_p = 0 \end{cases}$$

- We can write the Log Error equivalently - combining the two cases - in a single line:

$$\begin{aligned} g_p(w) &= -y_p \log(f(\mathrm{w}, x_p)) - (1 - y_p)\log\left(1 - f(\mathrm{w}, x_p)\right) \\ &= -y_p \log(\hat{y}\,) - (1 - y_p)\log\left(1 - \hat{y}\right) \end{aligned}$$

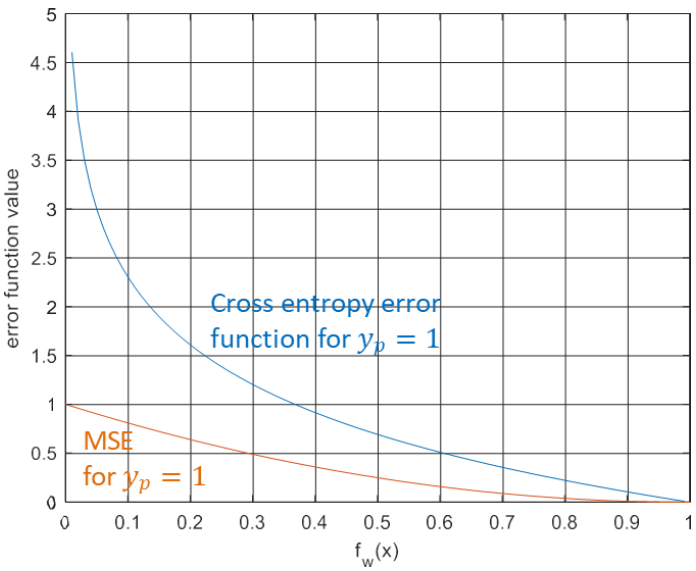
Loss Calculation: Log Error Vs MSE

$$g_p(w) = -y_p \log(f(\mathrm{w}, x_p)) - (1 - y_p) \log(1 - f(\mathrm{w}, x_p))$$

- This is the point-wise cross entropy error
  - Why is this error function better than the

    MSE $(f(w, x_p) - y_p)^2$?

    - The log error penalizes the violations of the desired output much more harshly than a squared error



EXAMPLE:

- Suppose we have *optimally tuned our weights* and that $y_p = 1$
- Then with our ideal weights we should satisfy our desired equality and have $\sigma(x_p^T w) \cong y_p = 1$, and if this indeed the case then have $-\log\left(\sigma(x_p^T w)\right) = -\log(1) = 0$
- However if $\sigma(x_p^T w) < 1$ the value $-\log\left(\sigma(x_p^T w)\right)$ *becomes large and positive very quickly harshly penalizing violations* of our desired equality
- As $\sigma(x_p^T w)$ approaches 0 - the worst possible value it could take on if indeed $y_p = 1$, then cost value goes to **positive infinity**
- So indeed, this point-wise cost function penalizes violations when $y_p = 1$ very strictly and is **minimal (equals 0)** in value when the desired equality holds

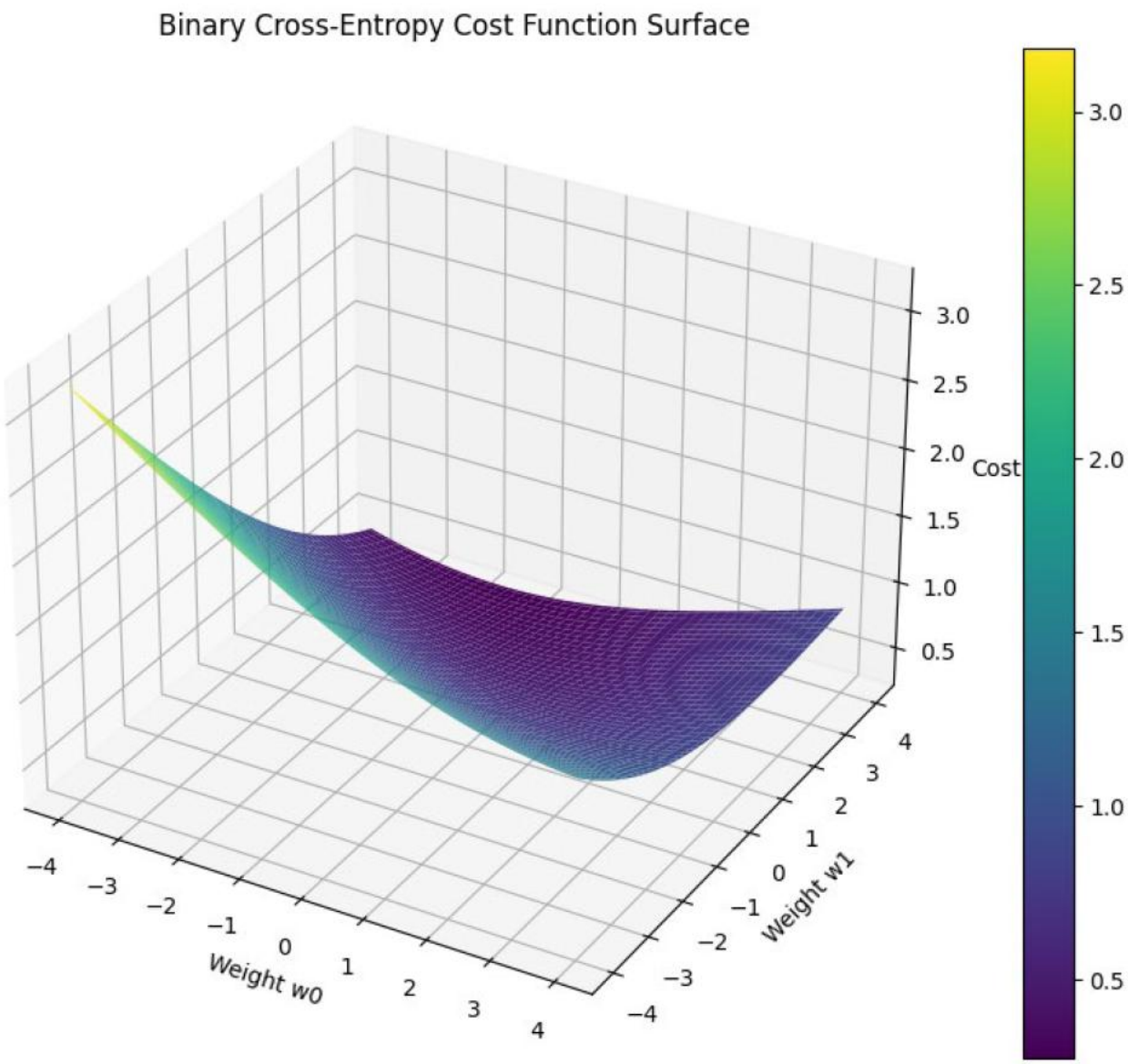
- In short, we always use $loss_p = -y_p \log(f(\mathrm{w}, x_p)) - (1 - y_p) \log (1 - f(\mathrm{w}, x_p))$
- This is simplified to $loss_p = -\log(probability\ of\ the\ true\ class)$
- Why we use $-\log$ in the loss?
  - Probabilities are between 0 and 1, and As probability → 0, log(probability) → -∞
  - What We Want:
    - Good prediction (high probability for correct class) → LOW loss
    - Bad prediction (low probability for correct class) → HIGH loss
    - Without the Negative Sign we will be trying to minimize a negative number, which means making it MORE negative (worse predictions = lower "loss").

- In **Logistic Regression** we use the so-called **Binary Cross Entropy cost function** by taking the average of the Log Error costs over all P points as:

$$g(w) = -\frac{1}{P}\sum_{p=1}^{P}\left[y_p \log f\left(w, x_p\right) + \left(1 - y_p\right)\log\left(1 - f\left(w, x_p\right)\right)\right]$$

  - $g(\boldsymbol{w})$ is the binary cross entropy error function (or cost function) over all datapoints
  - Note: P, x_p, and y_p are all given. Thus the error is a function of $\boldsymbol{w}$ only
  - The cross entropy error function $g(\boldsymbol{w})$ **is always convex**
  - To train the model, we tune $\boldsymbol{w}$ by minimizing $g(\boldsymbol{w})$
  - We can use different techniques such as Gradient Descent (more on this next class)

Surface Plot of g(w)



Training using Gradient Descent

Same as before

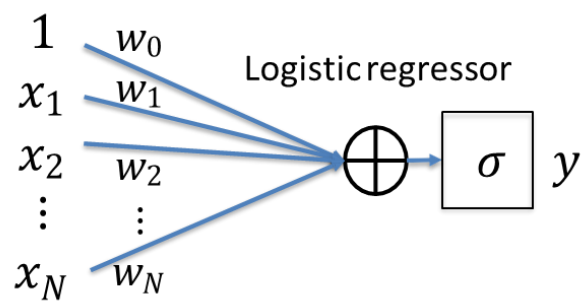
- Minimizing $g(\boldsymbol{w})$ can be done by gradient descent
- Gradient descent pseudocode:
  1. Initialize $k = 0$, and pick $\boldsymbol{w}^0$ as a starting point
  2. For step k, find $\nabla g(\boldsymbol{w}^k)$
     ($g(\boldsymbol{w})$ here is the cross-entropy cost function)
  3. Update $\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - \alpha \nabla g\left(\boldsymbol{w}^k\right)$ (for some step size α)
  4. Increment k
  5. Repeat 2-4 until stopping criterion (convergence or # iterations)

(More on this next class)

Logistic Regression with Multiple Features

- We still have C=2 classes
- Before N=1, Now let's see the case of N features in the input

- Dataset: P data points $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2)$, $(\boldsymbol{x}_P, y_P)$
  - $\boldsymbol{x}_p$ is p input data point (aka feature vector) with N features
    $\boldsymbol{x}_p = (\boldsymbol{x}_{1,p}, \boldsymbol{x}_{2,p}, \dots, \boldsymbol{x}_{N,p})$
  - Output: $y_p \in \{0,1\}$

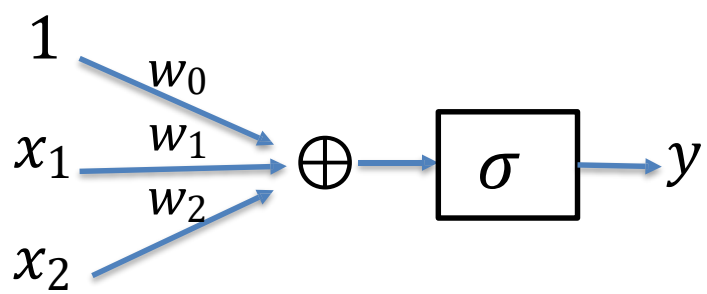


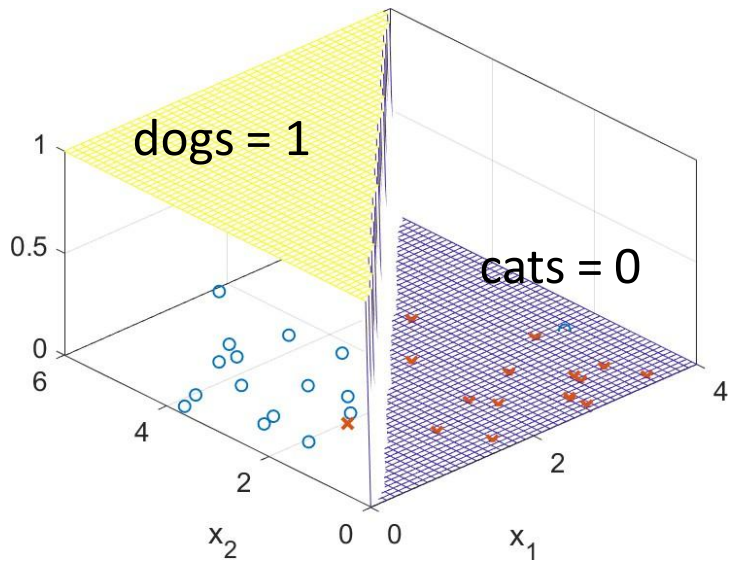
- Task: Find a function
  $f(w, \boldsymbol{x}_p) = \sigma(w_0 + w_1\, x_{1,p} + w_2\, x_{2,p} + \cdots + w_N\, x_{N,p})$
  such that $y_p \approx f(\mathrm{w}, \boldsymbol{x}_p)$
- To tune $\boldsymbol{w}$, we minimize

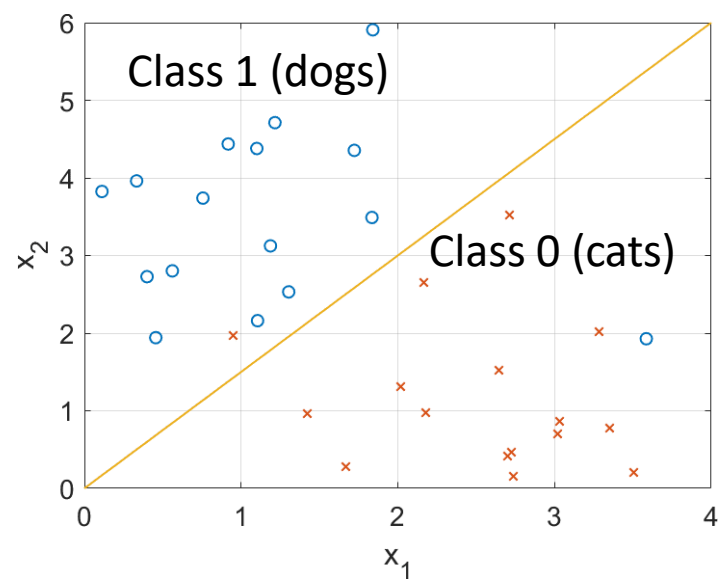
$$g(w) = -\frac{1}{P}\sum_{p=1}^{P} y_p \log(f(\mathrm{w},\boldsymbol{x}_p)) + (1 - \mathrm{y}_p)\log(1 - f(\mathrm{w},\boldsymbol{x}_p))$$

Example

- Let's work on an N=2 example
- P data points $(\boldsymbol{x}_1, y_{1)}, (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_P, y_P)$
- $\boldsymbol{x}_p = (\boldsymbol{x}_{1,p}, \boldsymbol{x}_{2,p})$ where $\boldsymbol{x}_{1,p}$ is the nose size and $\boldsymbol{x}_{2,p}$ is the ear shape
- Output: $y_p \in \{0,1\}$ (cat=0, dog=1)







- By tuning $\boldsymbol{w}$, we tune the boundary
- Example:

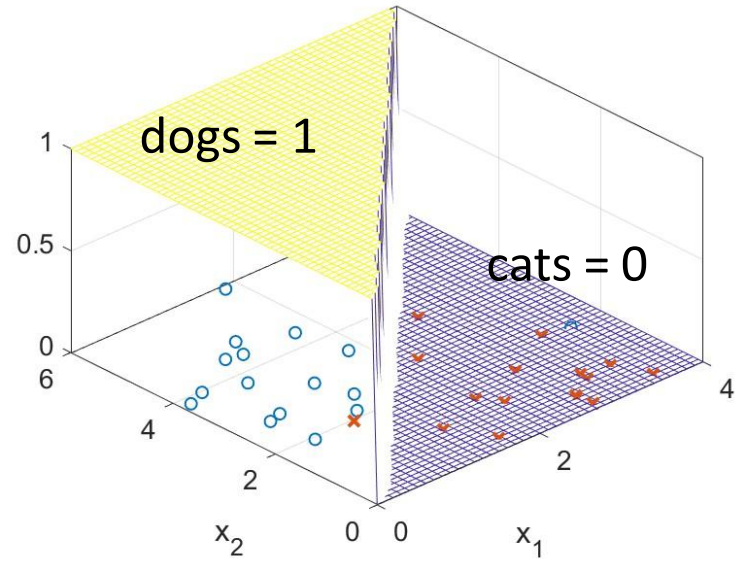
$$\boldsymbol{w} = (w_0, w_1, w_2) = (0.1,\ -1.5, 1)$$

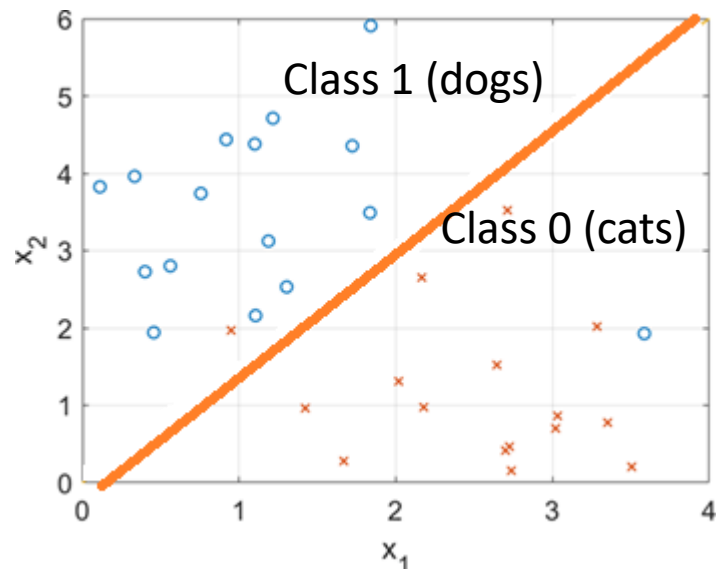
$\Rightarrow$ boundary is characterized by the line $w_0 + w_1 x_1 + w_2 x_2 = 0$, hence:

$$x_2 = -\frac{w_0 + w_1 x_1}{w_2} = -0.1 + 1.5 x_1$$

- Logistic regression with >2 features: Boundary is a hyperplane characterized by

$$w_0 + w_1 x_1 + \cdots + w_N x_N = 0$$



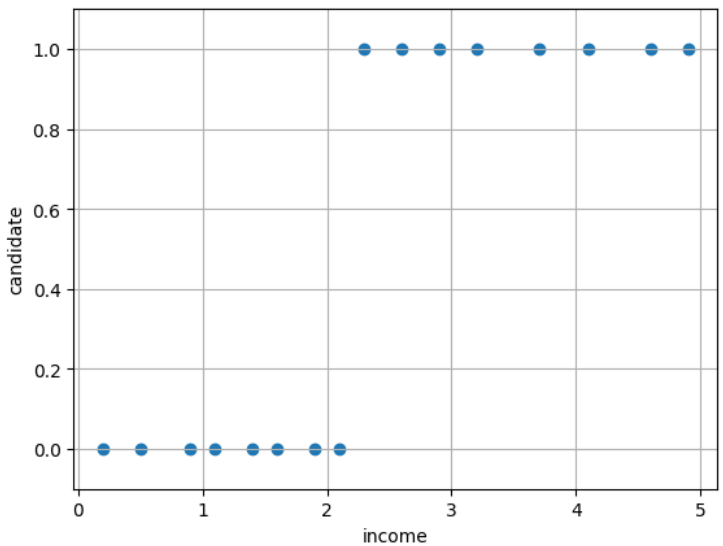


Logistic Regression Example

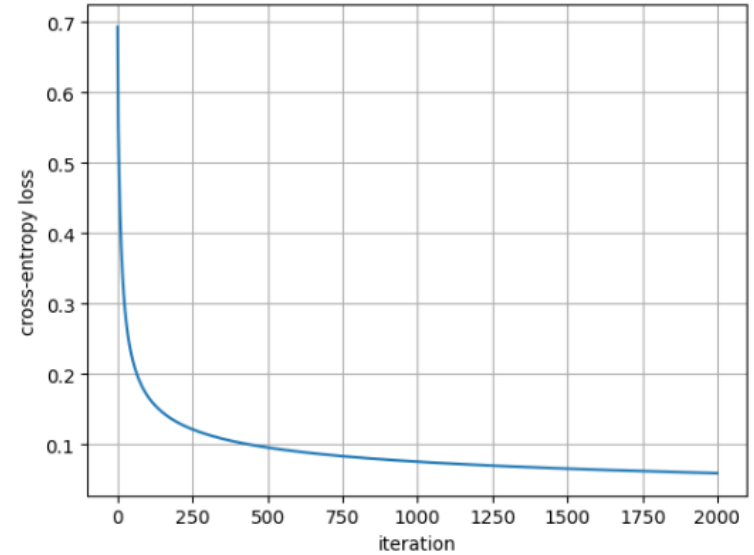
This is an N=1 Example (one feature in the input) and C=2 classes

```
############################## CELL 1
import numpy as np
import matplotlib.pyplot as plt

############################## CELL 2
#X = "income", just 1 feature, y = "candidate" (0 or 1)
X = np.array([0.2, 0.5, 0.9, 1.1, 1.4, 1.6, 1.9, 2.1, 2.3, 2.6, 2.9, 3.2, 3.7, 4.1, 4.6, 4.9]).reshape(-1, 1)
Y = np.array([0,   0,   0,   0,   0,   0,   0,   0,   1,   1,   1,   1,   1,   1,   1,   1]).reshape(-1, 1)
plt.scatter(X, Y)
plt.xlabel("income")
plt.ylabel("candidate")
plt.ylim(-0.1, 1.1)
plt.grid(True)
plt.show()

############################## CELL 3
def model(X, w):
   return w[0] + X * w[1] #w is shape (2,1): w[0]=bias, w[1]=weight
#caluclate the activation (probability)
def sigmoid(t):
    return 1.0 / (1.0 + np.exp(-t))
#caluclate the loss
def cross_entropy(w, X, Y, eps=1e-12):
    #predicted probabilities (Y_hat)
    a = sigmoid(model(X, w))
    #cross entropy: -( Y log(a) + (1-Y) log(1-a) )
    loss = -np.mean(Y * np.log(a) + (1 - Y) * np.log(1 - a))
    return loss
```

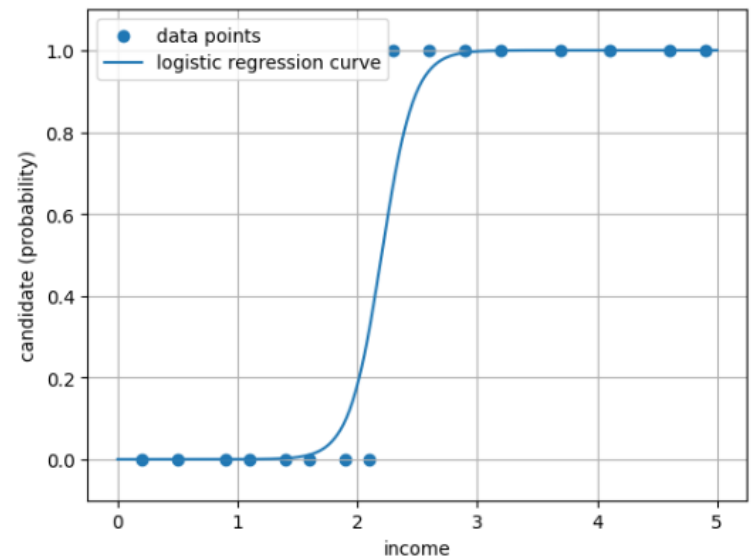
```python
############################## CELL 4
#Find the gradient based on current weights - single step at k iteration
def grad_cross_entropy(w, X, Y):
    P = X.shape[0]
    a = sigmoid(model(X, w))              #P×1
    error = a - Y                         #P×1
    dw0 = np.sum(error) / P               #scalar
    dw1 = np.sum(error * X) / P           #scalar
    grad = np.array([[dw0], [dw1]])       #2×1
    return grad

#The gradient descent algorithm - full iterative
def gradient_descent(X, Y, alpha=0.5, iters=2000):
    w = np.zeros((2,1))                   #random initialization , start at [0,0]
    loss_history = []
    for k in range(iters):
        loss = cross_entropy(w, X, Y)
        loss_history.append(loss)
        #find the gradient
        g = grad_cross_entropy(w, X, Y)
        w = w - alpha * g
        if k % 200 == 0:
            print(f"iter {k:4d} | loss = {loss:.6f} | w0={w[0,0]:.4f}, w1={w[1,0]:.4f}")
    return w, np.array(loss_history)
############################## CELL 5
#Start Training
w_final, loss_hist = gradient_descent(X, Y, alpha=1.0, iters=2000)
#plot loss
plt.plot(loss_hist)
plt.xlabel("iteration")
plt.ylabel("cross-entropy loss")
plt.grid(True)
plt.show()

#plot data + learned logistic curve
Xp = np.linspace(0, 5, 200).reshape(-1, 1)
Yp = sigmoid(model(Xp, w_final))

plt.scatter(X, Y, label="data points")
plt.plot(Xp, Yp, label="logistic regression curve")
plt.xlabel("income")
plt.ylabel("candidate (probability)")
plt.ylim(-0.1, 1.1)
plt.grid(True)
plt.legend()
plt.show()

print("\nFinal weights:")
print("w0 (bias) =", w_final[0,0])
print("w1        =", w_final[1,0])
```




```
... iter 1400 | loss = 0.067343 | w0=-14.3872, w1=6.5330
    iter 1600 | loss = 0.064259 | w0=-15.1018, w1=6.8575
    iter 1800 | loss = 0.061628 | w0=-15.7619, w1=7.1575
```




```
# to predict
w_ = w_final
x_test=1.2
print(sigmoid(model(x_test,w_)))

x_test=3.6
print(sigmoid(model(x_test,w_)))
```



OUTPUT

```
[0.00058037]
[0.99996942]
```

< 0.5: choose candidate 0

> 0.5: choose candidate 1

```
Final weights:
w0 (bias) = -16.37440918370628
w1        = 7.435948946866738
```